

Intro To Operating Systems

2026 Semester 1 SOFTENG 370: Operating Systems

Talia Xu

Lecture 1

2.0.1

Based on original slides by Professor Jon Eyolfson.

This work is licensed under a [Creative Commons Attribution-ShareAlike 4.0 International License](https://creativecommons.org/licenses/by-sa/4.0/)

About Me: My name is Talia



Lecturer

Computer Science

Research Interest:

Embedded Systems,
Low Power Wireless Communication,
IoT / Batteryless Systems.

I mostly teach **systems courses**,
operating systems, computer organization, etc.

Agenda

About the course

Introduction to Operating Systems

First topic: Process

Lecture Times and Location

Mon: 1pm - 2pm 303-G02

Tues: 1pm - 2pm 405-460

Wed: 1pm - 2pm 106-204

Lecture Slides: Generally available by the end of the day before each lecture.

Lecture Recordings: Posted by the end of the day of the lecture.

Practice Questions: Lectures will include practice questions similar to those you will see on the test and final exam.

Interactive Quizzes: Any interactive quizzes used in class will not be posted separately, but you can review them in the lecture recordings.

Your Instructors

Talia Xu

Instructor: Week 1 - 6

Office: 303S-592

Email: talia.xu@auckland.ac.nz

Office hour: by appointment

Shyamli Sindhwani

(Course Director/Coordinator)

Instructor: Week 7 - 12

Office: 303-409

Email: shyamli.sindhwani@auckland.ac.nz

Office hour: by appointment

Tutorial: Wed 3pm - 4pm 303-G01

Zhihang Liu

Email: zliu604@aucklanduni.ac.nz

Steve Le

Email: qlle752@aucklanduni.ac.nz

Format: Tutorials are a mix of reviews of course content, practice questions, and help sessions for assignments.

Resources: Tutorials are not recorded, but slides will be shared before each tutorial.

Assignments: TAs are also responsible for marking assignments. Please direct your assignment-related inquiries to them.

Evaluation for this course

Assessment	Weight	Due Date
Assignment 1	10%	March 29
Midterm	20%	April 21 @ 6:30 pm
Assignment 2	10%	TBD
Assignment 3	10%	TBD
Final	50%	TBD

Overall Mark: To pass the course, you must achieve at least 50% for your overall course mark.

Test/Exam Component: You must also pass the combined test/exam component. This test/exam component is worth 70% of the overall mark, consisting of the test (worth 20%) and the final exam (worth 50%). To pass this component, you need to get at least 35 out of 70.

Aegrotat / Compassionate Consideration Policy

If circumstances impacted your ability to sit an invigilated assessment, or impaired your preparation/performance, we will organise an alternative assessment.

The alternative assessment may be in the same format or in an alternative format such as viva voce/oral test.

The Recommended Books Complement Lectures

“Operating Systems: Three Easy Pieces”

by Remzi Arpaci-Dusseau and Andrea Arpaci-Dusseau

“Silberschatz’s Operating System Concepts, 10th Edition, Global Edition”

by Abraham Silberschatz and Peter B. Galvin and Greg Gagne

“The C Programming Language”

by Brian Kernighan and Dennis Ritchie

Skills You Should Practice Again If Needed

Programming

- C Programming and Debugging
- Python Programming and Debugging

Basic Concepts

- Being able to convert between binary, hex, and decimal
- Little-endian and big-endian
- Memory being byte-addressable, memory addresses (pointers)

Why Operating Systems?

Understanding the operating system will make you a better programmer

You will either write software that:

- Interacts with the operating system
- Is the operating system

Not always important if you only care about functionality.

Important when performance matters in your program.

Why Operating Systems?

```
int array[ROWS][COLS];  
for (int i = 0; i < ROWS; i++) {  
    for (int j = 0; j < COLS; j++) {  
        array[i][j] = i * COLS + j;  
    }  
}
```

```
int array[ROWS][COLS];  
for (int j = 0; j < COLS; j++) {  
    for (int i = 0; i < ROWS; i++) {  
        array[i][j] = i * COLS + j;  
    }  
}
```

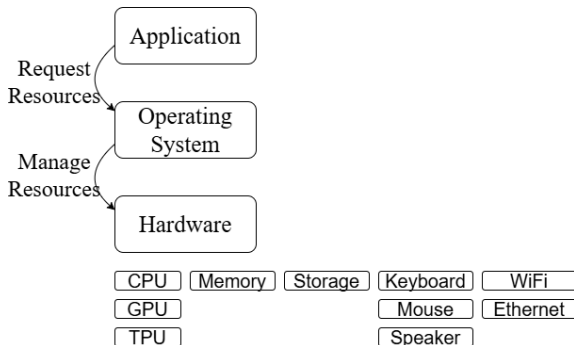
Why Operating Systems?

```
// 1 + 2 + 3 + 4 + 5 + ... + n = 100
```

```
int iterative_addition(int n) {  
    int sum = 0;  
    for (int i = 0; i < n; i++) {  
        sum += i;  
    }  
    return sum;  
}
```

```
int recursive_addition(int n) {  
    if (n == 0) return 0;  
    return (n - 1) + recursive_addition(n - 1);  
}
```

An Operating System Manages Resources



We have all used multiple OS

Windows

Linux

MacOS

Android

IOS

ChromeOS

....

Are they just the same OS with different names?

How are they different?

Three Core Operating System Concepts

Virtualization

lets each program think it has the whole computer to itself

Concurrency

allows multiple tasks to appear to happen at once

Persistence

guarantees that data will stick around and can be retrieved later

Our First Abstraction is a Process

Program: a file containing all the instructions and data required to run

Process: an instance of running a program

The Basic Requirements for a Process

How are you able to run two different programs at the same time?

For example, a “hello world” program and another that counts up one every second

Virtual Registers

Stack

Heap

Process

Does the OS Allocate Different Stacks For Each Process?

The stacks for each process need to be in physical memory

One option is the operating system just allocates
any unused memory for the stack

Would there be any issues with this?

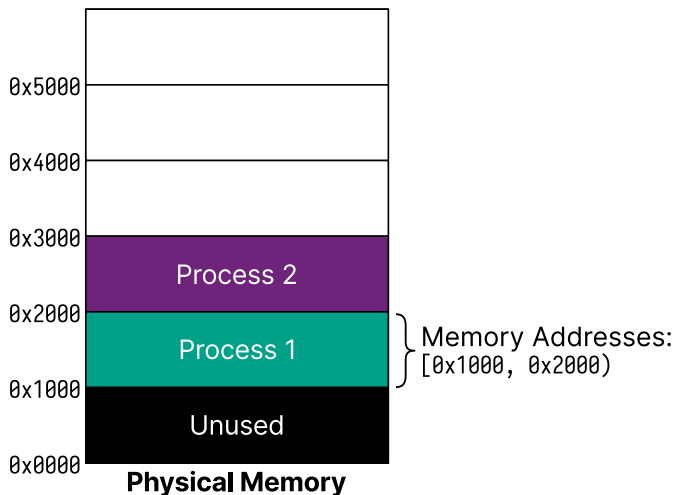
What About Global Variables?

The compiler needs to pick an address for each variable when you compile

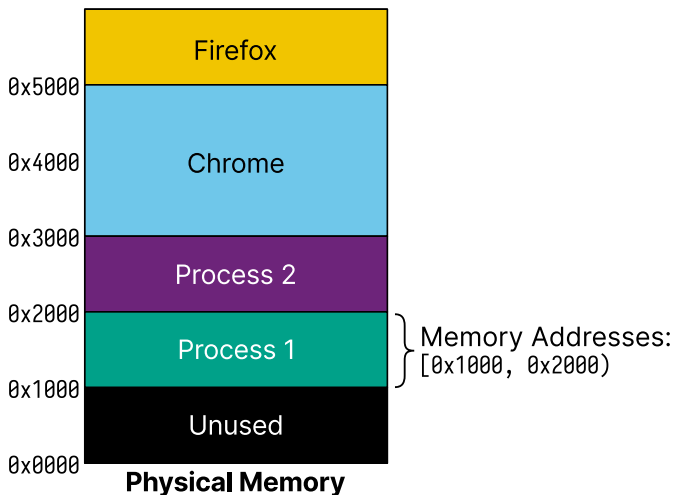
What if we had a global registry of addresses?

Would there be any issues with this?

Potential Memory Layout for Multiple Processes



Potential Memory Layout for Multiple Processes



What Happens If Two Processes Run the Same Program?

```
#include <stdio.h>
#include <unistd.h>

static int global = 0;

int main(void) {
    int local = 0;
    while (1) {
        ++local;
        ++global;
        printf("local = %d, global = %d\n", local, global);
        printf("0x%lx, 0x%lx\n", (long int)&local, (long int)&global);
        sleep(1);
    }
    return 0;
}
```

What Did We Find?

Was the address of `local` the same between the two processes?

Was the address of `global` the same between the two processes?

What else may be needed for a process?

A Process Has Its Own Virtual Memory

